

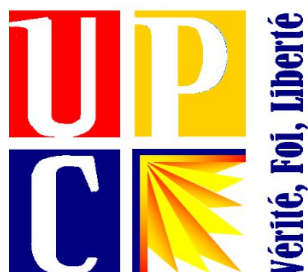
***UNIVERSITE PROTESTANTE AU CONGO***

**FACULTES DES SCIENCES INFORMATIQUES**

**L2 LMD FASI**

**B.P.4745**

**KINSHASA/LINGWALA**



**TP DE PHP:  
RAPPORT TECHNIQUE DE FASI CHAT  
CLASSROOM**

**Projet : FasiChat Classroom (Campus Relay)**

**Auteurs : THSIMANGA KALALA BLESS**

**MUMPUBI ELAM RUBIS**

**CIVAVA LITSANI ESTHER**

# Rapport Technique : Justification de la Conception Orientée Objet pour la Hiérarchie des Rôles Administratifs

## 1. Introduction

---

Dans le cadre du développement de la plateforme **FasiChat Classroom**, une application de messagerie académique interne pour la Faculté des Sciences de l'Information (FASI), le choix d'une architecture logicielle robuste et évolutive était impératif. Ce rapport technique justifie l'adoption de la **Conception Orientée Objet**, en mettant l'accent sur la modélisation de la hiérarchie des rôles administratifs et pédagogiques.

Le système doit gérer une diversité d'utilisateurs — du Doyen à l'étudiant — chacun possédant des responsabilités, des permissions et des interfaces de tableau de bord distinctes. La COO permet de traduire fidèlement ces relations du monde réel en une structure logicielle cohérente, facilitant ainsi la maintenance et l'extension future de la plateforme.

## 2. Rappel des Principes de la Programmation Orientée Objet (POO)

---

L'efficacité de la conception de FasiChat repose sur l'application rigoureuse des quatre piliers de la POO :

## 2.1. L'Encapsulation

L'encapsulation consiste à regrouper les données (attributs) et les méthodes qui les manipulent au sein d'une même entité appelée **classe**. Elle permet de protéger l'état interne d'un objet en n'autorisant l'accès que via des interfaces contrôlées (getters/setters).

## 2.2. L'Héritage

L'héritage permet de créer une nouvelle classe à partir d'une classe existante. La sous-classe hérite des attributs et méthodes de la classe parente (super-classe), favorisant la réutilisation du code et l'établissement d'une hiérarchie "est-un" (ex: un Doyen *est un* Utilisateur).

## 2.3. Le Polymorphisme

Le polymorphisme permet à des objets de classes différentes d'être traités de manière uniforme s'ils partagent une classe parente commune. Une même méthode (ex: `getPermissions`) peut avoir des implémentations différentes selon le type réel de l'objet, permettant une flexibilité dynamique.

## 2.4. L'Abstraction

L'abstraction permet de définir des modèles conceptuels sans se soucier des détails d'implémentation. Les classes abstraites servent de "contrats" imposant une structure commune à toutes les classes dérivées.

# 3. Analyse de la Structure Globale du Projet

---

Le projet FasiChat adopte une structure modulaire claire, séparant les préoccupations (Separation of Concerns) :

| Répertoire        | Description                                                                 |
|-------------------|-----------------------------------------------------------------------------|
| classes/modeles/  | Contient les définitions des entités métier (Utilisateurs, Messages, etc.). |
| classes/services/ | Gère les services transversaux comme la connexion à la base de données.     |
| config/           | Centralise les paramètres de configuration du système.                      |
| vues/             | Regroupe les fichiers de présentation (HTML/PHP).                           |

Cette organisation facilite la navigation dans le code et renforce la maintenabilité, chaque composant ayant une responsabilité unique et bien définie.

## 4. Modélisation de la Hiérarchie des Utilisateurs

---

Au cœur du système se trouve la classe abstraite `Utilisateur`. Cette classe définit le socle commun à tous les acteurs de la plateforme.

### 4.1. La Classe Abstraite `Utilisateur` : Le Socle Commun

La classe `Utilisateur` (`classes/modeles/Utilisateur.php`) utilise l'abstraction pour définir les propriétés universelles :

- **Attributs protégés** : `id`, `role`, `nomComplet`, `courriel`, `motDePasse`, etc.  
L'utilisation du modificateur `protected` permet aux classes filles d'accéder à ces données tout en les masquant pour le reste de l'application.
- **Méthodes concrètes** : Des fonctionnalités comme `hydrate()` (pour remplir l'objet à partir de la base de données) ou `verifierMotDePasse()` sont implémentées une seule fois pour éviter la redondance.
- **Méthodes abstraites** : Le système impose trois contrats que chaque rôle doit remplir :
  1. `getPermissions()` : Retourne la liste des actions autorisées.
  2. `peutEnvoyerMessageA()` : Définit les règles de communication.
  3. `getTableauBord()` : Structure les données spécifiques à l'interface de l'utilisateur.

## 4.2. Analyse des Rôles Administratifs

### 4.2.1. Le Doyen : L'Autorité Maximale

La classe `Doyen` étend `Utilisateur` en lui octroyant les privilèges les plus élevés.

- **Permissions :** `convoquer`, `vision_globale`, `gerer_utilisateurs`.
- **Communication :** Peut envoyer des messages à n'importe quel utilisateur du système sans restriction.

### 4.2.2. Le Vice-Doyen : La Gestion Académique

Le `ViceDoyen` possède des permissions similaires au `Doyen` mais orientées vers la gestion des cours.

- **Permissions :** `affecter_cours`, `convoquer`.
- **Communication :** Liberté totale de communication pour assurer la coordination pédagogique.

### 4.2.3. L'Apparitaire : Le Support Administratif

L'`Apparitaire` joue un rôle de diffuseur d'information.

- **Permissions :** `gerer_valve`, `publier_annonces`.
- **Communication :** Contrairement aux autres, sa méthode `peutEnvoyerMessageA()` retourne `false`. Ce choix de conception restreint l'Apparitaire à la diffusion via le "Valve" (panneau d'affichage), évitant ainsi de surcharger les messageries privées avec des flux non sollicités.

## 4.3. Analyse des Rôles Pédagogiques

### 4.3.1. L'Enseignant et l'Assistant

Ces classes partagent une logique de publication sur le “mur pédagogique”. L'Enseignant a toutefois la capacité de recevoir des convocations officielles du décanat.

### 4.3.2. L'Étudiant

Le rôle `Etudiant` illustre une règle métier complexe via le polymorphisme :

- **Règle de communication** : Un étudiant peut écrire à un enseignant, mais il ne peut écrire à un autre étudiant que s'ils appartiennent à la même **promotion**. Cette logique est encapsulée directement dans la méthode `peutEnvoyerMessageA(Utilisateur $destinataire)`.

## 5. Justification du Choix de la Conception Orientée Objet

---

L'utilisation de la POO pour la hiérarchie des rôles administratifs apporte des bénéfices tangibles au projet FasiChat :

### 5.1. Flexibilité Dynamique (Polymorphisme)

Grâce à la méthode fabrique (Factory Method) `createFromData` dans la classe `Utilisateur`, le système peut instancier le bon objet de rôle au moment de la connexion.

```
private static function createFromData(array $data): ?self
{
    $roleMap = [
        'doyen' => 'Doyen',
        'etudiant' => 'Etudiant',
        // ...
    ];
    $className = "\\Classes\\Modeles\\" . $roleMap[$data['role']];
    return new $className($data);
}
```

Cela permet au reste de l'application de manipuler un objet `Utilisateur` sans se soucier de son type réel. Par exemple, pour afficher un menu, on appelle `$utilisateur->getPermissions()` sans avoir besoin de multiples structures `switch` ou `if`.

## 5.2. Évolutivité du Système

Si la faculté décide demain d'ajouter un rôle "Secrétaire de Département", il suffit de créer une classe `Secretaire` héritant de `Utilisateur` et de définir ses permissions. Le code existant (messagerie, authentification) n'aura pas besoin d'être modifié pour intégrer ce nouveau rôle.

## 5.3. Sécurité et Intégrité des Données

L'encapsulation garantit que les règles de communication (ex: restriction étudiant-étudiant) ne peuvent pas être contournées facilement, car elles sont inscrites au cœur de l'objet lui-même.

## 5.4. Centralisation de la Logique Métier

Toute la logique liée à un rôle est regroupée dans un seul fichier. Cela facilite le débogage et garantit que les modifications de permissions sont appliquées de manière cohérente dans toute l'application.

# 6. Conclusion

---

La conception orientée objet adoptée pour le projet **FasiChat Classroom** dépasse la simple organisation du code. Elle constitue une véritable stratégie d'ingénierie logicielle permettant de gérer la complexité inhérente à une hiérarchie administrative académique.

En utilisant l'abstraction pour définir un cadre commun et le polymorphisme pour gérer les spécificités de chaque rôle (Doyen, Vice-Doyen, Apparitaire, etc.), la plateforme s'assure une longévité et une robustesse essentielles pour un outil de communication institutionnel. Ce choix architectural garantit que FasiChat peut évoluer au rythme des besoins de la Faculté des Sciences de l'Information, tout en restant un système fiable et facile à maintenir.

## 7. Références

---

1. **Dépôt GitHub du projet** : `Blesstk07/fasi_chat`
2. **Classes Modèles** : `Utilisateur.php` , `Doyen.php` , `ViceDoyen.php` , `Apparitaire.php`
3. **Services** : `BaseDeDonnees.php` (Pattern Singleton)
4. **Documentation PHP** : Modèles d'objets et héritage (php.net)